



CPLUS

- [1. Introducción](#)
- [2. Modelo Computacional](#)
 - [2.1. Dominio](#)
 - [2.2. Lenguaje](#)
- [3. Implementación](#)
 - [3.1. Frontend](#)
 - [3.2. Backend](#)
 - [3.3. Adicionales \(Opcional\)](#)
 - [3.4. Dificultades Encontradas](#)
- [4. Futuras Extensiones](#)
- [5. Conclusiones](#)
- [6. Referenecias](#)
- [7. Bilbiografía](#)

1. Introducción

El proyecto tiene como objetivo desarrollar un DSL (Domain specific language), junto con su compilador. De esta manera, se pasó por distintas fases: En primer lugar, se diseñó el lenguaje que se buscaba compilar. En segundo lugar, se implementó un analizador léxico y sintáctico. Por ultimo, se implementó el análisis semántico sobre lo construido.

2. Modelo Computacional

2.1. Dominio

El dominio de aplicación se centra en usar el paradigma orientado a objetos en un lenguaje de mejor performance y de mas bajo nivel como lo es C, el cual no pose cualidades de este paradigma. De esta manera, se traduce el lenguaje propuesto en lenguaje C, pudiendo así aprovechar la performance de C sin perder las grandes ventajas del OOP (Object-Oriented Programming).

2.2. Lenguaje

Lo principal que distingue este lenguaje del lenguaje C es la creación de clases, las cuales permiten encapsular datos y comportamiento. Se podría escribir un programa que contenga lo siguiente:

```
class simpleCalculator {
    public static int FORTY_TWO = 42;

    int sum(int num1, int num2) {
        return num1 + num2;
    }

    int sub(int num1, int num2) {
        return num1 - num2;
    }
}

int main() {
    simpleCalculator calculator = new simpleCalculator();
    int result = calculator.sum(10, 20);
    return calculator.sub(result, 5);
}
```

El output en C, sería el siguiente:

```
/* simpleCalculator_ADT.h */

typedef struct simpleCalculator_
CDT * simpleCalculator;

int simpleCalculator_sum(int num
1, int num2);

int simpleCalculator_sub(int num
1, int num2);
```

```
/* simpleCalculator_ADT.c */

#include "simpleCalculator_ADT.
h"

struct simpleCalculator_CDT {
};

int simpleCalculator_FORTY_TW
O = 42;

int simpleCalculator_sum(int num
1, int num2) {
    return (num1 + num2);
}
```

```
}

int simpleCalculator_sub(int num
1, int num2) {
    return (num1 - num2);
}
```

3. Implementación

3.1. Frontend

A la hora de empezar con el desarrollo del compilador lo primero que se realizó fue la definición de los tokens: por un lado, tomando como referencia proyectos que parsean C, se buscó una forma de parsear el **subconjunto** del lenguaje ya existente; por otro lado, hubo que idear el parseo y tokenización de nuevos símbolos, tales como *"new"*.

Esta primer etapa no trajo mayores dificultades, ya que no había que innovar demasiado con respecto a C.

Por otro lado, el diseño de la gramática en bison fue bastante más complejo. Se empezó pensando en como se estructuran los programas de C, y diseñando una abstracción para poder llevarlo a una gramática. Fue un proceso *agile*, dónde la gramática fue creciendo y modificándose a medida que se construía. Se pensó en los programas como listas de bloques, dónde un bloque podía ser una clase o una función clásica de C, dónde cada bloque se fue desarrollando más a detalle.

3.2. Backend

Para la etapa final, el primer objetivo fue únicamente recorrer todo el AST. Encontramos un par de bugs del frontend, que eran difíciles de reconocer sin analizar el árbol generado, entonces recurrimos a hacer un programa que haga una impresión del árbol. Por ejemplo, esto es una comparación de un resultado de un análisis sintáctico válido, y su correspondiente impresión del AST.

```
class myClass {
    private int a;

    constructor(int a) {
```

```

        this→a = a;
    }

    public int getA() {
        return this→a;
    }

    public void setA(int n) {
        this→a = n;
        return ;
    }
}

int main() {
    myClass obj = new myClass(10);
    obj→setA(20);
    return obj→getA();
}

```

```

--Program
|--BlockDeclaration (CLASS_BLOCK)
|  |-- "myClass"
|  |--ClassBody
|  |  |--MemberDeclaration (FIELD)
|  |  |  |--Access: private
|  |  |  |--Field: a : int
|  |  |  |--Type: int
|  |  |--MemberDeclaration (CONSTRUCTOR)
|  |  |  |--Method: (anon) → ??
|  |  |  |--Param: a : int
|  |  |  |--Type: int
|  |  |--Statement (EXPRESSION_STATEMENT)
|  |  |  |--Expression (ASSIGNMENT)
|  |  |  |  |--Expression (MEMBER_ACCESS) ""
|  |  |  |  |--Factor (CONSTANT)
|  |  |  |  |--Expression (IDENTIFIER_EXPRESSION) "a"
|  |  |--MemberDeclaration (METHOD)
|  |  |  |--Access: public

```

```

| | |--Method: getA → int
| | | |--Type: int
| | | |--Statement (RETURN_STATEMENT)
| | | | |--Expression (MEMBER_ACCESS) ""
| | | | |--Factor (CONSTANT)
| |--MemberDeclaration (METHOD)
| | |--Access: public
| | |--Method: setA → void
| | | |--Type: void
| | | |--Param: n : int
| | | | |--Type: int
| | | | |--Statement (EXPRESSION_STATEMENT)
| | | | | |--Expression (ASSIGNMENT)
| | | | | | |--Expression (MEMBER_ACCESS) ""
| | | | | | | |--Factor (CONSTANT)
| | | | | | |--Expression (IDENTIFIER_EXPRESSION) "n"
| | | |--Statement (RETURN_VOID_STATEMENT)
|--BlockDeclaration (METHOD_BLOCK)
|--Method: main → int
| |--Type: int
| |--Statement (INITIALIZED_DECLARATION_STATEMENT)
| | |--Type: myClass
| | |--Expression (NEW) "myClass"
| | | |--ArgumentList
| | | | |--Expression (INTEGER_EXPRESSION) 10
| |--Statement (EXPRESSION_STATEMENT)
| | |--Expression (FUNCTION_CALL) "<"()
| | | |--ArgumentList
| | | | |--Expression (INTEGER_EXPRESSION) 20
|--Statement (RETURN_STATEMENT)
| |--Expression (FUNCTION_CALL) "PE"()

```

Esto fue muy útil para debuggear y poder recorrer el árbol de manera correcta.

Una vez que logramos recorrer todo el AST, empezamos a pensar cómo implementar tabla de símbolos y el stack de scopes.

Como fue recomendado por la cátedra, implementamos en primer lugar un HashMapADT, el cual utilizaremos para almacenar las símbolos. También implementamos un StackADT para almacenar la información de los scopes.

Una vez que teníamos ambos, primero decimos que almacenaríamos las variables en un mapa, utilizando como clave el identificador de la variable (que por naturaleza es único), y como valor una estructura que contiene el tipo de dato, su inicialización y el identificador. Por otro lado, tendríamos el Stack de scopes, almacenando cuál es el scope actual. Nos dimos cuenta de que es innecesario guardar en el Stack únicamente el scopeld (int que identifica el stack). Aparte, dos variables podrían tener el mismo nombre si los scopes son distintos, lo que significa que la clave del mapa en realidad sería (identificador, scopeld).

Primero, se nos ocurrió un mapa de mapas, almacenando como primer clave el identificador de la variable, y como segunda clave un mapa cuya clave es el scopeld. Esto hacía muy eficiente la búsqueda de las variables declaradas en un scope como la búsqueda de si se puede o no usar una variable previamente declarada, pero tiene el downside de que al momento de popear un scope, no tengo almacenada la información invertida, es decir, no puedo eliminar todos los símbolos pertenecientes a un scope con complejidad $O(1)$.

Esto nos llevó a la decisión de tener un stack de mapas, almacenando, en cada scope, un mapa de los símbolos declarados en ese scope. Esto tiene la ventaja de inserción de nueva variable en el scope actual con complejidad $O(1)$, y permite eliminar todas las variables del scope popeado también con complejidad $O(1)$, pero pierde la ventaja de querer usar una variable previamente declarada, ya que hay que buscar en todos los scopes 'padres' si existe dicha declaración.

Para esto se utilizó una pila exterior donde ir desapilando. Esta fracción de código lo explica mejor:

```
StackADT pusher = createStack(sizeof(Scope*));
// _cs = compilerState
char * key = expression→identifier;
unsigned short found = 0;
while (!isEmptyStack(_cs→scopeStack) && !found) {
    Scope * scope = (Scope*) popScope(); // pops from main _cs→scopeStack
    HashMapADT symbols = scope→symbols;
    if (hash_map_get(symbols, &key) != NULL) {
        found = 1;
    }
}
```

```

pushStack(pusher, &scope);
}

//restoring
while (!isEmptyStack(pusher)) {
    Scope * scope;
    popStack(pusher, &scope);
    logDebugging(_logger, "Restoring scope id %d into the stack", scope->i
d);
    pushStack(_cs->scopeStack, &scope);
}

freeStack(pusher);
if (!found) {
    logError(_logger, "Unidentified symbol: %s", key);
    return _invalidComputation();
}
return _ok();

```

~~3.3. Adicionales (Opcional)~~

3.4. Dificultades Encontradas

La generación de código implicó generar un archivo header y un archivo de implementación, lo cual supuso la potencial apertura y manejo de múltiples archivos para un mismo programa. Pues si bien un programa puede contener múltiples definiciones de clase, para mantenerlas aisladas las implementaciones generadas deben encontrarse en archivos separados a la hora de compilar.

4. Futuras Extensiones

A futuro, el trabajo se podría potenciar incluyendo herencia de clases y tipos de datos genéricos, sobre los cuales también se podría realizar herencia y composición, permitiendo expresar relaciones o tipos de datos complejos.

5. Conclusiones

El desarrollo de este compilador nos permitió aplicar directamente los conceptos teóricos vistos en clase. La definición de la gramática fue el aspecto con mayor correlación teórica, permitiéndonos comprender en la práctica cómo ciertas reglas de producción pueden generar conflictos shift-reduce y ambigüedades.

El backend nos enfrentó a desafíos de diseño e implementación, particularmente en el manejo de memoria, la gestión de ownership entre módulos, y la elección de estructuras de datos para la tabla de símbolos evaluando sus trade-offs.

En resumen, este proyecto consolidó los conocimientos teóricos de la materia y nos brindó una comprensión profunda de los desafíos prácticos involucrados en la construcción de compiladores reales.

6. Referencias

7. Bibliografía

Agustin Golmar. *How to show abstract syntax tree of a grammar in bison?* Stack Overflow. https://stackoverflow.com/questions/8566430/how-to-show-abstract-syntax-tree-of-a-grammar-in-bison#comment139534034_8566598